# MAS - Homework 6
# Individual Homework

Iñigo Auza de la Mora (2824537)

22 December 2023

## 1 Monte Carlo Simulation

The COVID scare was a wake-up call, highlighting the vulnerability of our hyper-connected modern world to the threat of pandemics. Now that the COVID urgency has passed, governments across the globe are rushing to update their emergency action plans and disaster scripts.

As part of this effort, you've been hired by a medical team that has been tasked with developing fast procedures to detect a blood-borne virus. Since these tests need to be administered to large groups in the population, and testing resources are limited, the medics have come up with the following procedure. They started from the assumption that they need to test $N$ blood samples (of as many different individuals) and that $N$ is large (e.g. $N = 10^6$). Furthermore, the probability that an individual is infected is p, where p is relatively small, e.g. $p < 0.1$.

Based on these assumptions they propose the following procedure to minimise the number of tests they have to run: Rather than testing each sample individually, take a batch of k samples and mix them. Then this mixed sample is tested for the presence of the viral antigen:

- If the mixed sample tests negative (i.e. no viral antigen is detected), then all the individual samples were clear, and you therefore have the result for all k individual samples that went into the batch.
- If the mixed sample tests positive (i.e. the viral antigen is present indicating infection), one needs to retest all individual samples that went into the batch, in order to find out which individual(s) are actually infected.

**Questions:**

1. Use Monte Carlo simulation to estimate the optimal batch size k (i.e. the one that minimises the expected number of tests) for a given value of p where p can take values between $10^{-1}$ and $10^{-4}$.

   - This problem can visualized as a tree. However, it is important to notice that the tree would only have two layers, and every path has a single option (there is no bifurcation). Meaning, every branch has a single leave, and vice versa.
   - To explain the previous point, please consider our origin point to be our list of patients. From them, we have the opportunity to test the patients in batches of $k$ individuals. With $k$ ranging between 1 and $N$, the number of total individuals, which was fixed at $10^6$.
   - The rules of the setup indicate that once we have tested in k batches, we directly after test every individual that was found in a positive batch. In this manner, only one option derive for each k batch option. Please see the figure 1 for reference.

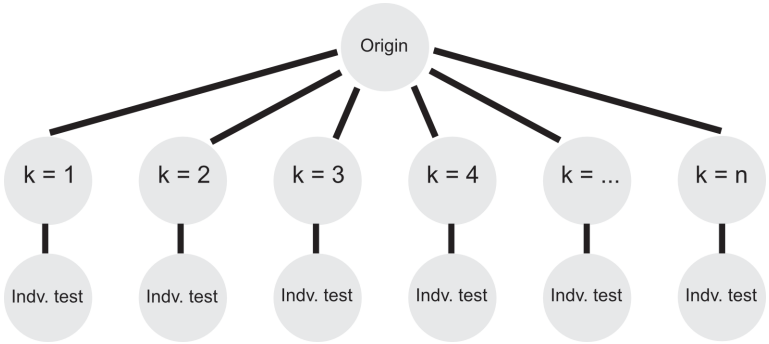



Figure 1: Representation of $k$-batch choosing as a tree

- As testing for batch $k = 1$ would result in testing every individual already, we will reduce this option to simply not using batching, where the number of tests is $N$.
- To test all values $k$, a list of patients is generated. During this procedure a value between $10^{-1}$ and $10^{-4}$ is generated, as the probability of a person being sick, for every patient. Followed by another random probability, with which it is decided if the patient is sick 1, or not 0.
- During every batch, the generated list of patients is be divided in batches size $k$, and continue the procedure. In this manner, we can re-use the initial list and divide it repeatedly to calculate all results for all $k$'s given a single, and equal, list of patients.
- It is important to notice that because this is a simple tree, it is also possible to not store a list of patients and instead utilize a seed when generating the random probabilities. This will result in the same patient list without having to store it. An example of this approach can be found in the code '1MonteCarloSimulation_V2.py'.
- After iterating through all possible $k$'s, a new list is produced, until a total of ten lists have been analyzed. Iterating through ten different patient list generated from the probabilities mentioned above resulted in the following:

| Run # | Optimal $k$ batch size | Number of tests needed |
|---|---|---|
| 1 | 4 | 593604 |
| 2 | 4 | 592376 |
| 3 | 4 | 593864 |
| 4 | 4 | 593756 |
| 5 | 4 | 593072 |
| 6 | 4 | 593436 |
| 7 | 4 | 594236 |
| 8 | 4 | 592792 |
| 9 | 4 | 594020 |
| 10 | 4 | 592592 |

Figure 2: Optimal $k$ batch throughout 10 individual runs

- These tests seem to suggest that 4 is the ideal batch size for the given sickness probabilities and $N$.

2. In order to convince your superiors that this a good investment, quantify the expected reduction in workload (compared to testing all samples individually).

   - After testing, the average minimum number of tests required was 593374.8 tests. Rounded to 593375, this would reflect an average reduction of 406625 tests. Requiring about 59% of the original amount of tests.

# 2 Thompson Sampling for Multi-Armed Bandits

## 2.1 Thompson's Bayesian update rule

Although the beta-distribution seems like a reasonable model to quantify the uncertainty on a probability, there is a deeper reason for its use. Updating a (prior) beta-density with binary observations, results in a new beta-density with updated parameters (this is an example of what is known as conjugated priors). Specifically, if the prior is modeled as $B(x, \alpha, \beta)$, and we observe s successes (1) and f failures (0) then the posterior would be the beta distribution $B(x; \alpha + s, \beta + f)$. This observation yields the rationale for Thompson's Bayesian update rule:

- Initialise $\alpha = \beta = 1$ (resulting in a uniform distribution, indicating that all possible values for p are equally likely). Now repeat the following loop:

  1. Sample from the bandit and get reward $r$ (either $r = 1$ or $r = 0$);
  2. Update the values for $\alpha$ and $\beta$ as follows:
     - if $r = 1$,then $\alpha \leftarrow \alpha + 1$
     - if $r = 0$,then $\beta \leftarrow \beta + 1$

**Questions:**

1. Implement the Thompson update rule for single arm bandit (i.e. k = 1) and show experimentally that the Beta-density increasingly peaks at the correct value for p. To this end, plot both the evolution of the mean and variance over (iteration) time.

   - To show this, the procedure above was followed over 300 epochs (iterations), logging in the change in $a$ and $b$. For this experiment, the chosen sample probability $p$ for the bandit's arm was 0.6. Figure 3 portrays the histogram of the mean beta-density, while figure 4 portrays the variance.
   - As expected, the value starts to settle near our probability $p = 0.6$.

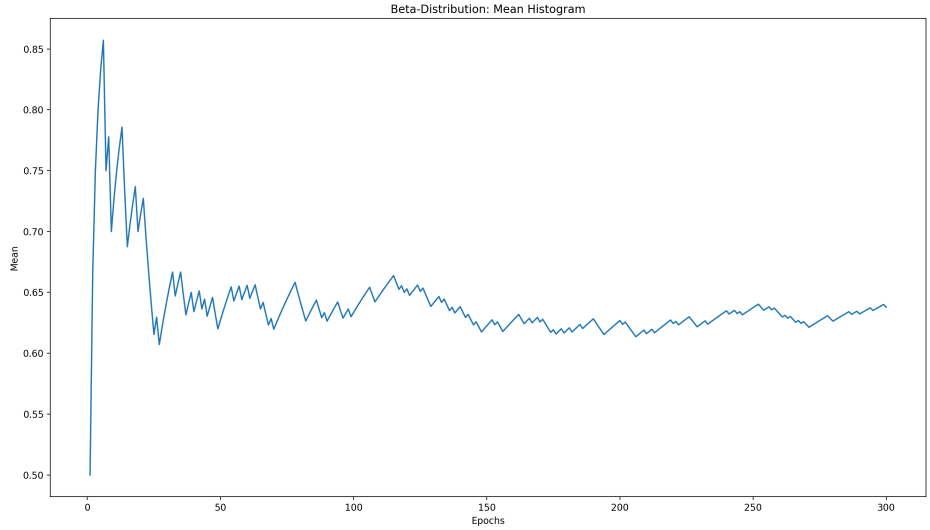



Figure 3: Mean beta-density histogram over 300 epochs

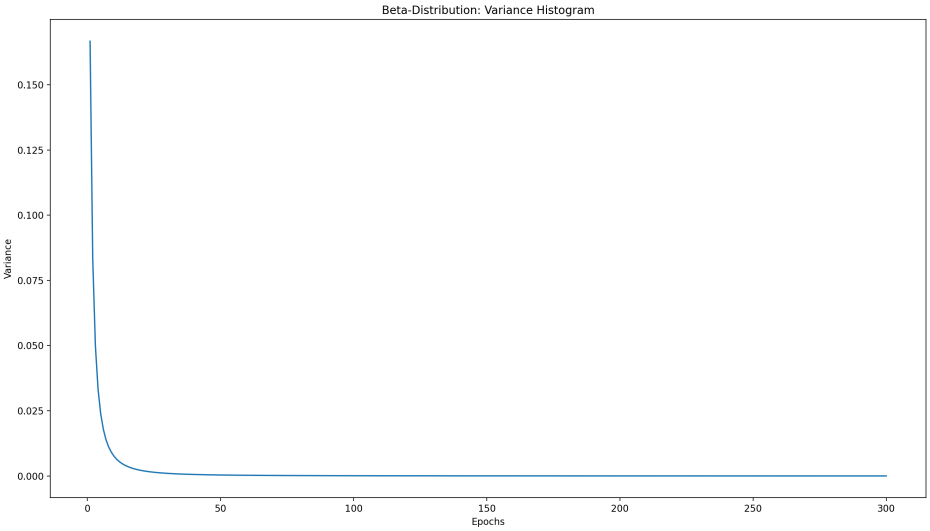



Figure 4: Variance beta-density histogram over 300 epochs

## 2.2 Thompson sampling for K-armed bandit Problem

For binary outcomes, the Thompson update rule offers an alternative for the UCB-based balancing of exploration and exploitation. Specifically, suppose we have a $K$-armed bandit problem. The $k$-th

arm delivers a reward $r = 1$ with (unknown!) probability $p_k$ (and hence $r = 0$ with probability $1 - p_k$). For each arm $(k = 1, ..., K)$, the uncertainty about the corresponding $p_k$ is modelled using a Beta-distribution $B(x; \alpha_k, \beta_k)$. Thompson sampling now tries to identify the arm that will deliver the maximal cumulative reward (highest $p_k$) by proceeding as follows:
Initialise all parameters to $1 : \alpha_k = 1 = \beta_k$; Now repeat the following loop:

- We use the beta-distributions to simulate the pulling of each arm. This means that we sample a value $U_k$ from each of the $K$ Beta-distributions:

$$U_k \sim B(x; \alpha_k, \beta_k)(k = 1...K).$$

- For this simulation, determine which arm gave the best result:

$$k_{max} = arg_{max} U_1, U_2, ..., U_K$$

- Mindful of the uncertainties on the $p_k$-values, the above simulation gives us reason to believe that pulling the $k_{max}$ arm is rational (after all, we did a simulation using the available evidence, and this was the result).

- Sample the corresponding arm (i.e. arm $k_{max}$) and get reward $r$ (either 1 or 0);

- Use the Bayesian update rule for the corresponding parameters:

$$\alpha_{k_{max}} \leftarrow \alpha_{k_{max}} + r \text{ and } \beta_{k_{max}} \leftarrow \beta_{k_{max}} + (1 - r)$$

**Questions:**

1. Write code to implement Thompson sampling for the above scenario when K = 3;

   - For this experiment, the arms were chosen to have a probability of .3, .7 and .4 of returning a reward of 1, respectively to each arm.
   - Ten unique runs, each with 300 epochs, were processed and can be seen in the following table figure 5.

| Run # | Factor | Arm 1 | Arm 2 | Arm 3 |
|---|---|---|---|---|
| 1 | a | 1 | 220 | 1 |
| | b | 2 | 80 | 1 |
| 2 | a | 3 | 218 | 1 |
| | b | 4 | 78 | 1 |
| 3 | a | 2 | 213 | 1 |
| | b | 3 | 85 | 1 |
| 4 | a | 1 | 202 | 1 |
| | b | 3 | 96 | 2 |
| 5 | a | 5 | 194 | 1 |
| | b | 6 | 98 | 1 |
| 6 | a | 1 | 1 | 117 |
| | b | 3 | 3 | 180 |
| 7 | a | 1 | 198 | 1 |
| | b | 4 | 98 | 3 |
| 8 | a | 9 | 186 | 1 |
| | b | 10 | 98 | 1 |
| 9 | a | 1 | 169 | 11 |
| | b | 3 | 99 | 22 |
| 10 | a | 4 | 219 | 1 |
| | b | 5 | 75 | 1 |

Figure 5: Thompson sampling over 10 iterations

- This resulted in the following average values for $a$ and $b$, and their corresponding beta-distribution.

| Factor | Arm 1 | Arm 2 | Arm 3 |
|---|---|---|---|
| Average a | 2.8 | 182.0 | 13.6 |
| Average b | 4.3 | 81.0 | 21.3 |
| beta-distribution | 0.3943662 | 0.6920152 | 0.3896848 |

Figure 6: Average Thompson sampling over 10 iterations

- It is expected that the beta distribution of the arm that we leverage the most will be the closest to its true value, as we have a larger data pool from which to approximate.
- It is important to notice that by initializing $\alpha$ and $\beta$ to 1, the mean beta distribution is 0.5. This results in an interesting behaviour where, because arm 2 has a higher probability of 0.7, and Thompson sampling focuses on the maximum expected utility, the sampling often ignores the arms beyond 2. We can see that arm 3 was not pulled in 6 oout of the 10 runs.

2. Perform numerical experiments in which you compare Thompson sampling with the UCB. Use total regret (provide the precise definition that you're using) as your performance criteria. For UCB, experiment with different values of the hyperparameter $c$. The fact that, for Thompson sampling, you don't need to specify an hyperparameter, is a distinct advantage.

- The Thompson sampling was compared to utilizing the Upper Confidence Bounds to select

an arm to sample (UCB). As the UCB contains a factor $c$, this was tested for $c = 1, 3, 5, 10$, individually. For this experiment, each sampling method was ran ten times each. The following table (figure 7) reflects the averages of those runs.

| Sampling | Factor | Arm 1 | Arm 2 | Arm 3 | Total Reward | Total Regret |
|---|---|---|---|---|---|---|
| Thompson | Average a | 2.8 | 182.0 | 13.6 | | |
| | Average b | 4.3 | 81.0 | 21.3 | 198.4 | 13 |
| | Avg beta-distrib. | 0.3943662 | 0.6920152 | 0.3896848 | | |
| UCB | Average a | 1.9 | 184.5 | 16.5 | | |
| $c = 1$ | Average b | 37 | 782 | 202 | 202.9 | 8.5 |
| | Avg beta-distrib. | 0.3392857 | 0.7023220 | 0.4495913 | | |
| UCB | Average a | 4.4 | 199.7 | 2.4 | | |
| $c = 3$ | Average b | 7.0 | 87.4 | 4.1 | 206.5 | 4.9 |
| | Avg beta-distrib. | 0.3859649 | 0.6955765 | 0.3692308 | | |
| UCB | Average a | 1.8 | 204.4 | 3.3 | | |
| $c = 5$ | Average b | 3.2 | 88.6 | 3.7 | 209.5 | 1.9 |
| | Avg beta-distrib. | 0.36 | 0.6976109 | 0.4714286 | | |
| UCB | Average a | 3.7 | 205.0 | 2.7 | | |
| $c = 10$ | Average b | 4.5 | 84.6 | 4.5 | 211.4 | 0 |
| | Avg beta-distrib. | 0.4512195 | 0.7078729 | 0.3750000 | | |

Figure 7: Average Thompson and UCB sampling over 10 iterations

- The total regret in figure 7 refers to the sum of all the regrets of the individual movements, as compared to the movements of the highest achieving sampling (UCB $c = 10$). This, is equivalent to the difference in rewards in the sum of all movements up to a certain time. In this example, all experiments were terminated in the epoch 300, making them simpler to compare to one another.

# 3 Reinforcement Learning: Cliff Walking

Consider the cliff-walking example (Sutton & Barto, ex. 6.6. p.108). Assume that the grid has 21 columns and 3 rows (above or in addition to the cliff). This is a standard undiscounted, episodic task, with start (S) and goal (G) states, and the usual actions causing movement up, down, right, and left. Reward is $-1$ on all transitions except:

- The transition to the terminal goal state (G) which has an associated reward of $+20$;
- transitions into the region marked The Cliff. Stepping into this region incurs a "reward" of $-100$ and also terminates the episode.

**Questions:**

1. Use both SARSA and Q-Learning to construct an appropriate policy. Do you observe the difference between the SARSA and Q-learning policies mentioned in the text (safe versus optimal path)? For Q-learning, experiment with a replay-buffer. Discuss.

   - As mentioned in the question, SARSA and Q-Learning present different behaviours, although this is also dependent on the epsilon $\epsilon$ value; but this will be discussed in the next question.

- As Q-Learning is an off-policy method, and only the max possible outcome of the movements of every state is accounted for, it becomes somewhat ignorant of the lower rewards. In this setup, it means it becomes "fearless", disregarding the greatly negative rewards in presence of other rewards. This encourages Q-Learning to find the shortest path to the highest rewards, accounting less for the negative values around it.
- In the other hand, the SARSA methodology includes all values, regardless if they are the maximum outcome or not, making it, in colloquial language, "more aware of possible dangers", and often finds paths that circumvent the large negative values.
- Meanwhile, utilizing a replay-buffer for the Q-Learning algorithm resulted in quickly reinforcing the path that was found to the goal. The outcome of this method depended on the epsilon $\epsilon$ value, but often resulted in the optimal path or similar.
- The tables were shorten to accommodate the spacing in this report. The experiment utilized the specified grid of 21 x 4. The seed for the random variables was fixed (random.seed(1)) to facilitate recreating the results. For the following, the parameters used remained $\epsilon = 0.1, \alpha = 0.3, \gamma = 0.7$ unless indicated otherwise.

  SARSA

```
['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

  Q-Learning

```
[0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

  Q-Learning with replay-buffering

```
[0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

- Understandably, these results may vary, as the methods are stochastic. Changing the random seed used in the calculations may results in variations. For example (with random.seed(6)):

  SARSA

```
['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 0.0]
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 'X', 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

Q-Learning

```
[0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

Q-Learning with replay-buffering

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0, 'X', 'X', 'X', ... 'X', 'X', 'X', 'X']
['X', 'X', 'X', 'X', 'X', 0.0, 0.0, ... 0.0, 0.0, 0.0, 'X']
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 'X']
```

2. Try different values for $\epsilon$ (parameter for $\epsilon$-greedy policy). How does the value of $\epsilon$ influence the result? Discuss.

   - The prominent change that can be noticed is in the behaviour of the SARSA implementation. As $\epsilon$ approaches 0, the behavior changes to optimal path finding, similar to the Q-Learning algorithm. The following results were calculated with the same random seed (random.seed(1)).

   $\epsilon = .05$

   ```
   ['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0]
   ['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 'X', 'X', 'X', 'X', 'X']
   ['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
   ['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
   ```

   $\epsilon = .005$

   ```
   [0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
   ['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 'X']
   ['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
   ['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
   ```

   $\epsilon = 0$

   ```
   [0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
   [0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
   ['X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 'X', 'X', 'X']
   ['X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
   ```

3. Now assume that there is a snake pit in the two bottom cells in the 11th column, i.e. the two cells closest to the cliff). Stumbling in these snake pits also carries a penalty (negative reward) of $-100$. Does this change the results obtained by SARSA or Q-learning? Discuss.

   - Naturally, as now there is an obstacle that also results in very low rewards (greatly negative), the implementations will circumvent it. This is expected given that such behavior will result in a higher total reward.

- However, it is important to notice that it is not specified if the snake pit is a terminal state. The behaviour of the implementations changes, and makes SARSA specially cautious if the snake pit is are terminal states.
- The following results were calculated utilizing $\epsilon = .005, \alpha = .3, \gamma = .7$ and the random seed random.seed(1), assuming that the snake pit are not terminal states.

SARSA

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X', 'X', ... 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', 'X', 'X', 'X', 0.0, ... 0.0, 0.0, 'X', 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

Q-Learning

```
[0.0, 0.0, 0.0, 0.0, 0.0, 'X', 'X', ... 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 'X', 'X', 'X', 0.0, ... 0.0, 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 'X', 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

Q-Learning with replay-buffering

```
[0.0, ... 0.0, 0.0, 'X', 'X', 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, ... 'X', 'X', 'X', 0.0, 0.0, 0.0, 'X', 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0]
['X', ... 'X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X', 'X', 'X', 'X', 'X', 'X']
['X', ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

- Meanwhile, when the states are marked as terminal, the SARSA behavior changes considerably. It appears to be specially "cautious" before crossing the snake pit, where all rewards are negative.
- In comparison, for Q-Learning the path gradually ascends, passing over the snake pits, to then gradually descend. Although the nature of this setup means that in this case all three solutions of these implementations receive the same reward and use the same amount of steps to reach the goal, the gradient created by the Q-Learning can be seen as an effort to remain tightly close to the optima path and the goal. Had there been diagonal steps, this may have resulted in a more efficient movement.
- For all three implementations, the overall behaviour resembles more that of question 1 .

SARSA

```
['X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', ... 'X', 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 'X', 'X', 'X', 'X', 0.0, 0.0]
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

Q-Learning

```
[0.0, 0.0, 0.0, 0.0, 'X', 'X', 'X', 'X', ... 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', 'X', 0.0, 0.0, 0.0, ... 0.0, 0.0, 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0]
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```

Q-Learning with replay-buffering

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X', 'X', ... 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X', 0.0, ... 0.0, 'X', 'X', 'X', 0.0, 0.0, 0.0, 0.0, 0.0]
['X', 'X', 'X', 'X', 'X', 'X', 'X', 0.0, ... 0.0, 0.0, 0.0, 'X', 'X', 'X', 'X', 'X', 'X']
['X', 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, ... 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 'X']
```